JUMPING MARIO

DISEÑO DE CÓDIGO

MELISA GUZMÁN MUÑOZ

DISEÑO DE DATOS Y ALGORITMOS

EDWARD ALEXANDER FRANCIA GUZMAN

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

BOGOTÁ, 2026

Descripción:

El programa recibe varios casos de prueba, en cada caso se tienen las alturas de N paredes organizadas de izquierda a derecha. Mario comienza en la primera pared y debe saltar consecutivamente hasta la última.

- Si la siguiente pared es **más alta** el salto es salto alto.
- Si la siguiente pared es **más baja** es salto bajo.
- Si son iguales no se cuenta.

El objetivo es calcular cuántos saltos altos y cuántos saltos bajos realiza Mario en cada caso.

Requerimientos:

RF1: El sistema debe leer un número T de casos.

RF2: Para cada caso debe leer un número N de paredes.

RF3: Debe leer las N alturas.

RF4: Debe contar los saltos altos.

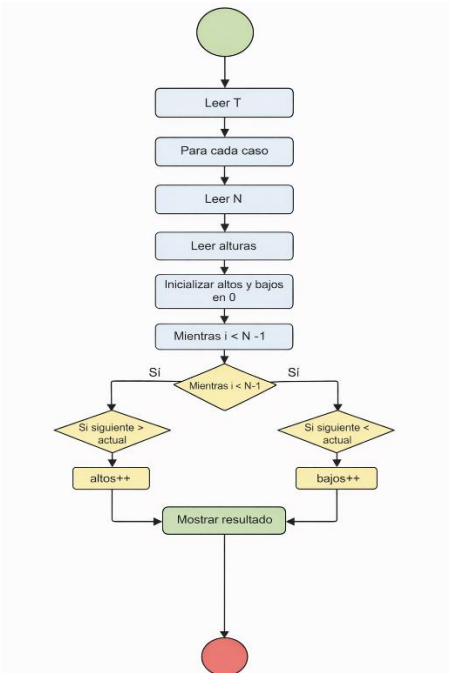
RF5: Debe contar los saltos bajos.

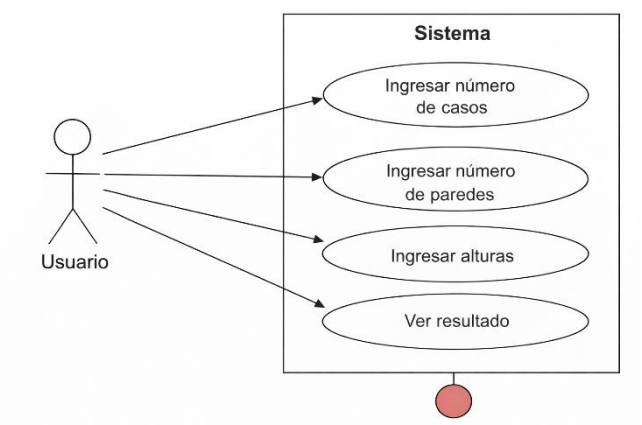
RF6: Debe mostrar el resultado.

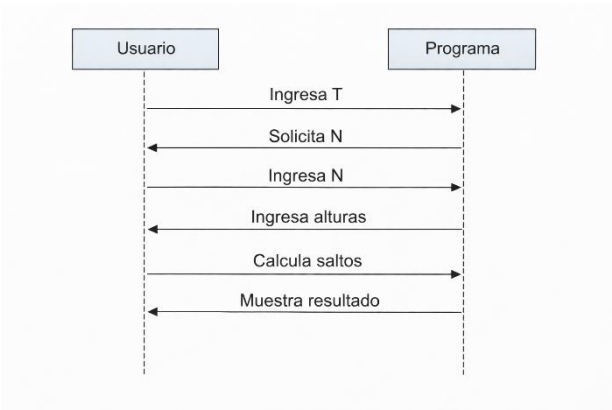
Historias de usuario:

- HU1: Como usuario, quiero ingresar varios casos de prueba para evaluar diferentes escenarios.
- HU2: Como usuario, quiero que el sistema me diga cuántos saltos altos y bajos hubo.
- HU3: Como usuario, quiero que los resultados estén organizados por número de caso.

Diagramas:


Leer T
Para cada caso
Leer N
Leer alturas
Inicializar altos y bajos en 0
Mientras i < N -1
Sí
Mientras i < N-1
Sí
Si siguiente > actual
Si siguiente < actual
altos++
bajos++
Mostrar resultado


Sistema
Ingresar número de casos
Ingresar número de paredes
Ingresar alturas
Ver resultado
Usuario


Usuario
Programa
Ingresa T
Solicita N
Ingresa N
Ingresa alturas
Calcula saltos
Muestra resultado

}Análisis de complejidad:

Nº	Instrucción	Costo	Veces ejecutada	Total
1	N = int(input())	c1	1	c1
2	alturas = input().split()	c2	1	c2
3	i = 0	c3	1	c3
4	comparación i < N	c4	N + 1	c4(N + 1)
5	alturas[i] = int(alturas[i])	c5	N	c5N
6	i = i + 1	c6	N	c6N
7	altos = 0	c7	1	c7
8	bajos = 0	c8	1	c8
9	i = 0	c9	1	c9
10	comparación i < N-1	c10	N	c10N
11	comparación alturas[i+1] > alturas[i]	c11	N - 1	c11(N - 1)
12	comparación alturas[i+1] < alturas[i]	c12	N - 1	c12(N - 1)
13	incremento altos o bajos	c13	N - 1	c13(N - 1)
14	i = i + 1	c14	N - 1	c14(N - 1)
15	print(resultado)	c15	1	c15

Ecuación: $T(N) = (c4 + c5 + c6 + c10 + c11 + c12 + c13 + c14)N + (c1 + c2 + c3 + c4 + c7 + c8 + c9 + c15 - c11 - c12 - c13 - c14)$

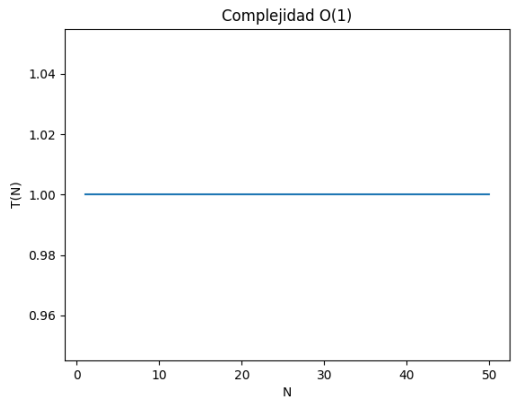
Mejor caso:

Para qye N fuera =21 el problema debería tener solo una pared (no habría problema en sí)

$$T(1) = c1 + c2 + c3 + c4(2) + c5(1) + c6(1) + c7 + c8 + c9 + c10(1) + c11(0) + c12(0) + c13(0) + c14(0) + c15$$

$$T(1) = c1 + c2 + c3 + 2c4 + c5 + c6 + c7 + c8 + c9 + c10 + c15$$

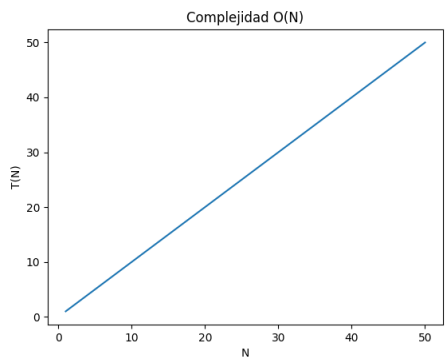
Todos son términos constantes por ende $T(\mathrm{n}) = O(1)$



Peor caso:

$$T(N) = (c4 + c5 + c6 + c10 + c11 + c12 + c13 + c14)N \\ + (c1 + c2 + c3 + c4 + c7 + c8 + c9 + c15 - c11 - c12 - c13 - c14)$$

El algoritmo es estrictamente lineal.



Código:

```python
T = int(input("ingrese el número de casos"))

for caso in range(T):
    N = int(input("ingrese el número de paredes"))
    alturas = input("ingrese las alturas").split()

    i = 0
    while i < N:
        alturas[i] = int(alturas[i])
        i = i + 1

    altos = 0
    bajos = 0

    i = 0
    while i < N - 1:
        if alturas[i + 1] > alturas[i]:
            altos = altos + 1
        elif alturas[i + 1] < alturas[i]:
            bajos = bajos + 1
        i = i + 1

    print("Case", caso + 1, ":", altos, bajos)
```

Tests:

```python
def contar_saltos(alturas):
    altos = 0
    bajos = 0
    i = 0
    while i < len(alturas) - 1:
        if alturas[i + 1] > alturas[i]:
            altos += 1
        elif alturas[i + 1] < alturas[i]:
            bajos += 1
        i += 1
    return altos, bajos


# TESTS
assert contar_saltos([1, 4, 2, 2, 3]) == (2, 1)
assert contar_saltos([4, 3, 2, 1]) == (0, 3)
assert contar_saltos([1, 1, 1, 1]) == (0, 0)
assert contar_saltos([1, 2, 3, 4]) == (3, 0)

print("Todos los tests pasaron correctamente")
```